

ATS Resume Builder

Complete Setup Guide

Google OAuth 2.0 · OpenRouter AI · Django Configuration

Step-by-step instructions to get every API key and configure your app

■ URL to visit

■■ Value to type / paste

■■ Important warning

Table of Contents

Part 1 — Google OAuth 2.0 Credentials

Google Cloud Console → OAuth 2.0 Client ID

Part 2 — Enable Google Drive API

Add Drive scope to your OAuth consent screen

Part 3 — OpenRouter API Key

openrouter.ai → free account → API key

Part 4 — Configure .env File

Paste all keys into your local .env

Part 5 — Django Admin — Link OAuth App

Social Applications in /admin/

Part 6 — Test Everything

Checklist to verify each integration works

Part 7 — Vercel Production Deployment

Add secrets to Vercel dashboard

Part 1: Google OAuth 2.0 Credentials

Get `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`

Google OAuth lets users sign in with their Google account and grants access to Google Drive. You need two values: a Client ID and a Client Secret. Both come from Google Cloud Console.

■ Open your browser and go to: <https://console.cloud.google.com>

1

Create or select a project

Click the project dropdown at the top-left of the Google Cloud Console.

Click **NEW PROJECT**, give it a name (e.g. "ATS Resume Builder"), then click **Create**.

Wait for the project to be created, then select it from the dropdown.

2

Open the OAuth consent screen

In the left sidebar go to: **APIs & Services** → **OAuth consent screen**

Choose **External** (allows any Google account to sign in) and click **Create**.

3

Fill in the App Information form

App name: ATS Resume Builder

User support email: your email address

App logo: optional — skip for now

Developer contact information: your email address again

Click **Save and Continue**.

4

Add OAuth Scopes

Click **Add or Remove Scopes**.

Search for and select these three scopes:

✓ `.../auth/userinfo.email`

✓ `.../auth/userinfo.profile`

✓ `.../auth/drive.file`

Click **Update**, then **Save and Continue**.

5

Add Test Users (while app is in Testing mode)

Click **Add Users** and enter your own Gmail address.

This lets you test login before publishing the app.

Click **Save and Continue** → **Back to Dashboard**.

6

Create the OAuth 2.0 Client ID

Go to: **APIs & Services** → **Credentials**

Click **+ Create Credentials** → choose **OAuth client ID**.

Application type: Web application

Name: ATS Resume Builder (or any name)

7

Add Authorised Redirect URIs

Scroll down to the **Authorised redirect URIs** section.

Click + **Add URI** and add the following (both are needed):

`http://127.0.0.1:8000/accounts/google/login/callback/`

`http://localhost:8000/accounts/google/login/callback/`

For production, also add:

`https://YOUR-APP.vercel.app/accounts/google/login/callback/`

Click **Create**.

8

Copy your credentials

A popup shows your **Client ID** and **Client Secret**.

Click **Download JSON** to save a backup, OR

Copy each value and paste them into your `.env` file (see Part 4).

■■ Never share or commit these values to GitHub.

Warning: If you see "Access blocked" during testing, make sure your Gmail address is added as a Test User in Step 5 and that the redirect URIs in Step 7 exactly match what your server is running on.

Part 2: Enable Google Drive API

Required for saving resumes to Google Drive

The Google Drive API must be explicitly enabled in your project before the OAuth drive.file scope will work. This is a one-time step per project.

1

Open the API Library

In Google Cloud Console left sidebar go to: **APIs & Services** → **Library**

2

Search for Google Drive API

Type "Google Drive API" in the search box and press Enter.

Click the **Google Drive API** result card.

3

Enable it

Click the blue **Enable** button.

Wait a few seconds for it to activate.

You will be taken to the API overview page — that means it worked.

4

Verify (optional)

Go to **APIs & Services** → **Enabled APIs & Services**.

"Google Drive API" should appear in the list with status Enabled.

■ You do NOT need a separate service account for this project. The app uses each user's own Google account (via OAuth) to access their Drive. Files are stored in a folder called "ATS Resume Builder" in the user's Drive.

Part 3: OpenRouter API Key

Get `OPENROUTER_API_KEY` for the AI Career Assistant

OpenRouter is a single API that gives access to many AI models (GPT-4o, Claude, Gemini, Llama, and more). The free tier includes credits to get started. Your key is kept server-side — it is never sent to the browser.

■ Open your browser and go to: <https://openrouter.ai>

1

Create a free account

Click **Sign Up** at the top-right.

You can sign up with Google, GitHub, or email.

Verify your email if prompted.

2

Go to the API Keys page

After logging in, click your avatar (top-right) → **Keys**.

Or go directly to: <https://openrouter.ai/settings/keys>

3

Create a new key

Click **Create Key**.

Name: ATS Resume Builder (or anything descriptive)

Credit limit: leave blank (unlimited from your balance) or set a monthly cap.

Click **Create**.

4

Copy the key immediately

The key is shown only once — copy it now.

It starts with: `sk-or-v1-...`

Paste it into your `.env` file (see Part 4).

5

Add credits (optional for free models)

Click **Credits** in the sidebar.

Some models (e.g. `meta-llama/llama-3.1-8b-instruct`) are free with \$0 balance.

Add a small amount (\$5) to unlock GPT-4o mini and other low-cost models.

The default model in this app is **openai/gpt-4o-mini** (~\$0.15 per 1M tokens).

■ Free alternative: set `OPENROUTER_DEFAULT_MODEL=meta-llama/llama-3.1-8b-instruct` in your `.env`.
This model is free (rate-limited) and works well for resume tasks.

Part 4: Configure Your .env File

Paste all keys into the local .env file

The `.env` file lives in the root of your project (same folder as `manage.py`). It is listed in `.gitignore` so it is never committed to GitHub. Open it with any text editor and fill in all values.

■ File location: C:\Users\srini\Desktop\projects\New folder\.env

Complete .env template — fill every line:

```
# ■■ Django Core ██████████  
SECRET_KEY=django-insecure-REPLACE-with-a-long-random-string  
  
DEBUG=True  
  
ALLOWED_HOSTS=localhost,127.0.0.1  
  
# ■■ Google OAuth (from Part 1) ██████████  
GOOGLE_CLIENT_ID=123456789-abcdefghijklmnop.apps.googleusercontent.com  
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxxxxxxxxxxx  
  
# ■■ OpenRouter AI (from Part 3) ██████████  
OPENROUTER_API_KEY=sk-or-v1-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx  
OPENROUTER_DEFAULT_MODEL=openai/gpt-4o-mini  
  
# ■■ Email (optional – console backend works for local dev) ■■■  
EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend  
  
# ■■ Production only (leave blank for local dev) ████████  
# DATABASE_URL=postgresql://user:pass@host:5432/dbname
```

Generate a secure SECRET KEY

Run this command once in your terminal:

```
venv\Scripts\activate
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

Copy the output and paste it as the SECRET KEY value.

■ ■ Never set `DEBUG=True` in production. Never commit `.env` to GitHub. The `.gitignore` in this project already excludes it.

Part 5: Django Admin — Link the OAuth App

Tell Django about your Google credentials via `/admin/`

Django-allauth stores OAuth provider credentials in the database (not just env vars). The management command `configure_google_oauth` does this automatically, but you can also do it manually from the admin panel.

Option A — Automatic (recommended)

After filling in `.env`, run these commands in your terminal:

```
venv\Scripts\activate
python manage.py migrate
python manage.py seed_data
python manage.py configure_site
python manage.py configure_google_oauth
python manage.py runserver
```

■ `configure_google_oauth` reads `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` from `.env` and creates the Social Application automatically. You will see "Google OAuth app created/updated" in the terminal.

Option B — Manual (via Admin panel)

1

Open Django Admin

Start the server: `python manage.py runserver`

Go to: `http://127.0.0.1:8000/admin/`

Log in with: `admin@admin.com` / `admin123`

2

Go to Social Applications

In the left sidebar, find **Social Accounts** → **Social Applications**.

Click **Add Social Application**.

3

Fill in the form

Provider: Google

Name: Google OAuth

Client id: paste your `GOOGLE_CLIENT_ID` value

Secret key: paste your `GOOGLE_CLIENT_SECRET` value

Key: leave blank

4

Link to the Site

In the **Sites** section at the bottom, move "example.com" (or "127.0.0.1:8000") from "Available sites" to "Chosen sites" by double-clicking it.

Click **Save**.

Part 6: Test Everything

Verification checklist after setup

Run through this checklist after completing Parts 1–5 to confirm everything is working.

#	Check	Expected result
1	Server starts cleanly	<code>python manage.py runserver</code> — no errors, no warnings about missing settings
2	Landing page loads	<code>http://127.0.0.1:8000/</code> → page renders with correct theme
3	Email signup works	<code>/accounts/signup/</code> → fill form → should redirect to <code>/dashboard/</code>
4	Email login works	<code>/accounts/login/</code> → email + password → redirects to <code>/dashboard/</code>
5	Google button is visible	<code>/accounts/login/</code> → Google button appears (not the yellow warning banner)
6	Google OAuth flow completes	Click "Continue with Google" → Google asks for permission → redirects to <code>/dashboard/</code>
7	Resume create works	<code>/dashboard/resumes/create/</code> → fill title → Create → builder page opens
8	PDF download works	In builder → Download PDF → file downloads (not a 500 error)
9	ATS check works	<code>/ats/</code> → paste a job description + select resume → Run Check → score appears
10	AI chat responds	<code>/ai/</code> → type a message → response appears (error if <code>OPENROUTER_API_KEY</code> is wrong/missing)
11	Google Drive upload works	In builder → Upload to Drive → success toast (only if signed in via Google)
12	Admin panel accessible	<code>http://127.0.0.1:8000/admin/</code> → login → all models visible

Common errors and fixes

Error	Fix
Error 400: <code>invalid_request</code> — Missing <code>client_id</code>	<code>GOOGLE_CLIENT_ID</code> is empty in <code>.env</code> , or <code>configure_google_oauth</code> was not run.
Error 403: Access denied	Your Gmail is not added as a Test User in the OAuth consent screen (Step 5 of Part 1).
Error 401 from OpenRouter	<code>OPENROUTER_API_KEY</code> is wrong or missing in <code>.env</code> .

Google button still shows warning banner	Server was not restarted after editing .env. Stop and restart python manage.py runserver.
Drive upload fails with "No Google credentials"	User signed in via email, not Google. Drive requires Google OAuth login.

Part 7: Vercel Production Deployment

Add all secrets to Vercel dashboard — never hard-code them

When deploying to Vercel, environment variables are set in the Vercel dashboard — not in `.env` (which stays local). Vercel injects them at build time and runtime.

1 Create a PostgreSQL database

Recommended: Neon.tech (free tier — 512 MB)

Sign up at <https://neon.tech> → New Project → copy the connection string.

It looks like: `postgresql://user:pass@ep-xxx.neon.tech/dbname?sslmode=require`

2 Push code to GitHub

```
git init
```

```
git add .
```

```
git commit -m "Initial commit"
```

```
git remote add origin https://github.com/YOUR-USERNAME/ats-resume-builder.git
```

```
git push -u origin main
```

3 Import project to Vercel

Go to <https://vercel.com> → New Project.

Click Import on your GitHub repository.

Framework Preset: Other

Build Command: `bash build_files.sh`

Output Directory: `staticfiles`

Do NOT deploy yet — add env vars first.

4 Add Environment Variables in Vercel

Click **Environment Variables** tab before deploying.

Add each variable from the table below.

5 Update Google redirect URI

Back in Google Cloud Console → Credentials → your OAuth Client.

Add a new Authorised Redirect URI:

`https://YOUR-APP.vercel.app/accounts/google/login/callback/`

Click Save.

6 Deploy

Click **Deploy** in Vercel.

Watch the build log — `build_files.sh` runs migrations, seeds data, configures OAuth.

When finished, visit `https://YOUR-APP.vercel.app`

7 Update the Django Site domain

Go to `https://YOUR-APP.vercel.app/admin/`

Log in → Sites → click "example.com" → change domain to `YOUR-APP.vercel.app`

Click Save. This fixes allauth callback URLs in production.

Vercel Environment Variables — complete list

Variable	Value	Notes
----------	-------	-------

SECRET_KEY	Long random string — generate with Django command	Required
DEBUG	False	Required
ALLOWED_HOSTS	YOUR-APP.vercel.app	Required
DATABASE_URL	postgresql://... from Neon.tech	Required
GOOGLE_CLIENT_ID	From Google Cloud Console (Part 1)	For Google login
GOOGLE_CLIENT_SECRET	From Google Cloud Console (Part 1)	For Google login
OPENROUTER_API_KEY	sk-or-v1-... from openrouter.ai (Part 3)	For AI features
OPENROUTER_DEFAULT_MODEL	openai/gpt-4o-mini	Optional
EMAIL_BACKEND	django.core.mail.backends.smtp.Email Backend	For email auth
EMAIL_HOST	smtp.gmail.com	Optional
EMAIL_HOST_USER	your-email@gmail.com	Optional
EMAIL_HOST_PASSWORD	Gmail App Password	Optional

■■ Security reminder: Vercel encrypts environment variables at rest. Never put secrets in vercel.json or any file committed to GitHub. Rotate your keys if they are ever accidentally exposed.